Linux Permissions

Day 1 Investigation Worksheet — detailed responses

rwx

What did I ask?	What do r, w, and x represent in Linux file permissions?
What did AI tell me?	r represents read permission, w represents write permission, and x represents execute permission. For directories, read allows listing entries, write allows modifying entries, and execute allows accessing entries within the directory when other required permissions are present.
What did I expect?	A permission system that controls whether users can read, modify, execute files, or access directories.
What command/action did I perform?	ls -l; touch test.txt; chmod 644 test.txt; ls -l test.txt
What happened?	Inspected file permissions, changed the test file's permissions, and verified the resulting permission string.
Yes / No / Partially (AI correctness)	Yes
What did I learn?	Linux uses read, write, and execute permissions to control access to files and directories. Understanding these permissions helps diagnose access errors and apply appropriate restrictions.

Owner, group, and others

What did I ask?	What do owner, group, and others mean in Linux file permissions?
What did AI tell me?	The owner is the user associated with the file's ownership. The group is the group assigned to the file, and others refers to users who do not qualify as the owner or members of that group for permission evaluation.
What did I expect?	Three categories used by Linux to determine which permissions apply to different users accessing a file or directory.
What command/action did I perform?	ls -l test.txt; id
What happened?	Inspected the file's owner, group, and permission string using ls -l. Used id to display the current user's UID and group memberships.
Yes / No / Partially (AI correctness)	Yes
What did I learn?	Linux evaluates permissions according to file ownership and the user's relationship to the owner and group. These categories help control access without granting every user identical permissions.

chmod 755

What did I ask?	What does the command chmod 755 do in Linux?
What did AI tell me?	chmod 755 sets the owner's permissions to read, write, and execute (rwx), while the group and others receive read and execute permissions (r-x). Read = 4, write = 2, and execute = 1.

What did I expect?	A command that grants full permissions to the owner while allowing other users to read and execute the file or access a directory.
What command/action did I perform?	touch test.txt; chmod 755 test.txt; ls -l test.txt
What happened?	Changed the file's permissions to 755 and verified that the permission string became -rwxr-xr-x, assuming the operation succeeded.
Yes / No / Partially (AI correctness)	Yes
What did I learn?	chmod 755 is commonly used for executable files and directories that should be accessible to other users. It allows the owner to modify the file while preventing group members and others from writing to it.

chmod 644

What did I ask?	What does the command chmod 644 do in Linux?
What did AI tell me?	chmod 644 grants the owner read and write permissions (rw-), while the group and others receive read-only permissions (r--).
What did I expect?	A command that lets the owner modify a file while allowing other users to read it without granting write or execute permissions.
What command/action did I perform?	touch test.txt; chmod 644 test.txt; ls -l test.txt
What happened?	Set the file's permissions to 644 and verified that the permission string became -rw-r--r--, assuming the operation succeeded.
Yes / No / Partially (AI correctness)	Yes
What did I learn?	chmod 644 is commonly suitable for ordinary files that should be readable by others but writable only by the owner. It does not grant execute permission.

Why chmod 777 is discouraged

What did I ask?	Why is chmod 777 generally discouraged in Linux?
What did AI tell me?	chmod 777 grants read, write, and execute permissions to the owner, group, and others. This can allow any local user with access to modify or execute the file, creating unnecessary security risks.
What did I expect?	A permission setting that allows everyone to access, modify, and execute a file or access and modify a directory, even when those permissions are unnecessary.
What command/action did I perform?	Inspected file permissions using ls -l and examined the meaning of the 777 permission value.
What happened?	Determined that 777 grants all three permission types to all three user categories, potentially exposing files and directories to unauthorized modification or misuse.
Yes / No / Partially (AI correctness)	Yes
What did I learn?	chmod 777 should generally be avoided because it grants excessive permissions. I should identify the actual cause of a permission error and grant only the minimum permissions required.

Challenge: Permission denied

What did I ask?	Is chmod 777 always required when a user encounters a Permission denied error in Linux?
What did AI tell me?	No. The error can result from incorrect ownership, insufficient file or directory permissions, missing execute permission on a directory, or other access restrictions. The solution depends on the cause.
What did I expect?	A troubleshooting approach that identifies the actual permission issue instead of granting unrestricted access to everyone.
What command/action did I perform?	ls -l <path>; id; namei -l <path>
What happened?	These commands can inspect file permissions, user and group memberships, and permissions along the path to the target. They help identify where access is being denied.
Yes / No / Partially (AI correctness)	Yes
What did I learn?	chmod 777 is not a universal fix for permission errors. Inspect ownership and permissions, identify the intended access requirements, and apply the smallest necessary change.

Final Learning

The investigation helped me understand Linux permission categories and numeric permission values. The safest approach is to inspect ownership and permissions, then grant only the access required instead of using chmod 777 as a blanket fix.